TREINAMENTO CYPRESS · SERVEREST

DIA 2

Construindo uma suíte

02

O que vamos fazer hoje

- **2.1** Retomada
- **2.2** Alias
- **2.3** Testes de API com `cy.request`
- **2.4** `cy.intercept` e login via API
- **2.5** Testes data driven
- **2.6** Comandos customizados
- **2.7** Evidências e plugins
- **2.8** Lab

Três perguntas do Dia 1

1. Por que `const email = cy.get('...').invoke('val')` não guarda o texto?

2. Por que `cy.wait(3000)` é problema mesmo quando o teste passa?

3. O que acontece ao rodar só o segundo teste com `it.only`?

2.2

Alias

Alias é um apelido

```
beforeEach(() => {
  cy.visit('/login')
  cy.get('[data-testid="email"]').as('campoEmail')
})

it('deve preencher os campos', () => {
  cy.get('@campoEmail').type('admin.treinamento@qa.com')
})
```

O `@` diz *"isto é um apelido, não um seletor"*.

Alias de dados

```
beforeEach(() => {
  cy.fixture('usuarios').as('usuarios')
})

it('deve logar com administrador', function () {
  cy.visit('/login')
  cy.get('[data-testid="email"]').type(this.usuarios.admin.email)
})
```

Repare: `function () {}`, não arrow function.

Lembram da semente de ontem?

É agora

Troque para arrow function

```
it('deve logar com administrador', () => {
  cy.visit('/login')
  cy.get('[data-testid="email"]').type(this.usuarios.admin.email)
})
```

```
Cannot read properties of undefined (reading 'usuarios')
```

Arrow function **não tem `this` próprio**.
E o alias mora no `this` do teste.

A alternativa

```
it('deve logar com administrador', () => {
  cy.get('@usuarios').then((usuarios) => {
    cy.visit('/login')
    cy.get('[data-testid="email"]').type(usuarios.admin.email)
  })
})
```

Funciona igual. Use a que você achar mais legível.

Alias de requisição

O mais importante dos três

Esperar um acontecimento

```
cy.intercept('POST', '**/login').as('requisicaoLogin')

cy.get('[data-testid="entrar"]').click()

cy.wait('@requisicaoLogin')
  .its('response.statusCode')
  .should('eq', 200)
```

A diferença

	Espera o quê	Problema
`cy.wait(3000)`	Um **tempo**	Chute
`cy.wait('@login')`	Um **acontecimento**	Fato

Mesmo nome. Naturezas opostas.

Esperar tempo é adivinhação

Esperar acontecimento é engenharia

E você ganha de brinde

```
cy.wait('@requisicaoLogin').then((intercepcao) => {
  expect(intercepcao.response.statusCode).to.eq(200)
  expect(intercepcao.response.body.authorization).to.contain('Bearer')
  expect(intercepcao.request.body.email).to.eq('admin.treinamento@qa.com')
})
```

Dá para validar a requisição, não só esperá-la.

2.3

Testes de API

Por que testar API

	Pela tela	Pela API
Tempo	Segundos	Milissegundos
Estabilidade	Renderização, animação, rede	Só a resposta
Cobertura	Só o que tem botão	Qualquer regra

O que isso NÃO significa

Não substitui teste de interface.

O usuário usa a tela, não a API.

Mas boa parte do que testamos clicando poderia ser testado muito mais barato.

Leitura simples

```
const apiUrl = Cypress.env('apiUrl')

it('deve listar usuários', () => {
  cy.request('GET', `${apiUrl}/usuarios`).then((res) => {
    expect(res.status).to.eq(200)
    expect(res.body).to.have.property('quantidade')
    expect(res.body.usuarios).to.be.an('array')
  })
})
```

Criação

```
const usuario = {
  nome: 'Usuário Treinamento',
  email: `treino_${Date.now()}@qa.com`,
  password: 'teste123',
  administrador: 'true'
}

cy.request('POST', `${apiUrl}/usuarios`, usuario).then((res) => {
  expect(res.status).to.eq(201)
  expect(res.body.message).to.eq('Cadastro realizado com sucesso')
})
```

O caminho negativo

```
const usuario = { nome: 'Usuário Duplicado', email: `duplicado_${Date.now()}@qa.com`, ... }
cy.request('POST', `${apiUrl}/usuarios`, usuario)     // cadastra

cy.request({
  method: 'POST',
  url: `${apiUrl}/usuarios`,
  body: usuario,                                      // e tenta de novo
  failOnStatusCode: false
}).then((res) => {
  expect(res.status).to.eq(400)
  expect(res.body.message).to.eq('Este email já está sendo usado')
})
```

O e-mail repetido é do próprio teste. **O do admin, nunca.**

`failOnStatusCode: false`

Por padrão o Cypress falha em status de erro

Para testar **de propósito** o caminho
que dá errado, você desliga

Autenticação encadeada

```
cy.request('POST', `${apiUrl}/login`, {
  email: 'admin.treinamento@qa.com', password: 'treino123'
}).then((login) => {
  const token = login.body.authorization

  cy.request({
    method: 'POST',
    url: `${apiUrl}/produtos`,
    headers: { authorization: token },
    body: { nome: `Produto ${Date.now()}`, preco: 100, ... }
  })
})
```

A estrutura

```
pedir token
  └─ quando chegar
       └─ usar o token na próxima requisição
```

O segundo `cy.request` está **dentro** do `.then()` do primeiro.

Ele só pode existir depois que o token chegou.

Repetitivo, não é?

Guardem isso

2.4

`cy.intercept` e login via API

Três usos, em escala crescente

1. **Observar** — espiar e esperar

2. **Substituir** — responder no lugar do servidor

3. **Forçar o impossível** — simular o que você não consegue provocar

Antes deles, uma ferramenta: **entrar sem passar pela tela**

Login via API

O padrão do resto do treinamento

A porta dos fundos

```
function entrarViaApi() {
  cy.request('POST', `${apiUrl}/login`, {
    email: 'admin.treinamento@qa.com', password: 'treino123'
  }).then((res) => {
    cy.visit('/login')          // estabelece a origem
    cy.window().then((win) => {
      win.localStorage.setItem('serverest/userToken', res.body.authorization)
      win.localStorage.setItem('serverest/userNome', 'Administrador do Treinamento')
    })
  })
}
```

🔴 O `cy.visit('/login')` antes não é desperdício

É ele que estabelece a origem do `localStorage`.

Sem isso, grava no lugar errado — e o teste volta para o login sem dizer por quê.

A conta

Trinta testes fazendo login pela tela.
Três segundos cada.

Um minuto e meio só nisso.

E trinta chances de falhar por um motivo que não tem nada a ver com o que o teste deveria validar.

Teste o login uma vez

Nos outros trinta, entre pela porta dos fundos

Uso 1 · Espiar e esperar

```
cy.intercept('GET', '**/produtos*').as('listagemProdutos')

entrarViaApi()          // tela do admin: sem sessão, volta ao /login
cy.visit('/admin/listarprodutos')

cy.wait('@listagemProdutos').its('response.statusCode').should('eq', 200)
```

A armadilha

Registre o intercept ANTES do `cy.visit`
Ele só captura o que passar depois de existir. Registrado depois, o `cy.wait` estoura sem explicar.

E o `*` no fim de `**/produtos*`?
Casa também com `/produtos?nome=x`. Aqui a listagem não usa query string — é hábito de proteção.

Uso 2 · Simular a resposta

```
cy.intercept('GET', '**/produtos*', {
  fixture: 'produtos-mock.json'
}).as('produtosMock')

entrarViaApi()
cy.visit('/admin/listarprodutos')
cy.wait('@produtosMock')

cy.get('table.table-striped tbody tr').should('have.length', 2)
```

O ganho é **determinismo**.

Lista vazia

```
cy.intercept('GET', '**/produtos*', {
  body: { quantidade: 0, produtos: [] }
}).as('semProdutos')

entrarViaApi()
cy.visit('/admin/listarprodutos')
cy.wait('@semProdutos')

cy.get('table.table-striped tbody tr').should('have.length', 0)
```

O cenário que todo mundo esquece e que quebra em produção.

A tela não avisa nada

Tabela em branco, sem uma linha de texto

Quem encontrou foi o **teste**

Como você testaria a tela de erro 500 sem derrubar o servidor?

Assim

```
cy.intercept('GET', '**/produtos*', {
  statusCode: 500,
  body: {}
}).as('erroServidor')
```

O servidor está ótimo.

A sua aplicação acha que ele caiu.

E a aplicação quebra

Não mostra erro bonito. Quebra mesmo.

Por quê

O ServeRest faz:

```
.catch(e => console.error(e))
```

E segue adiante tentando ler os dados de uma resposta que não veio.

Você acabou de encontrar um defeito real.

Agora a decisão

```
// Esconder — o teste passa e o bug some
Cypress.on('uncaught:exception', () => false)
```

```
// Afirmar — o teste passa e o bug fica registrado
const erros = []
cy.on('uncaught:exception', (err) => {
  erros.push(err.message)
  return false
})
// ...
cy.wrap(erros).should((lista) => {
  expect(lista.join(' ')).to.contain('undefined')
})
```

As duas deixam o teste verde

Só uma conta a verdade

O perigo do global

Um `uncaught:exception => false` no arquivo de suporte faz **todos** os erros da aplicação sumirem.

Inclusive os que você ainda não descobriu.

É silenciar o alarme de incêndio porque ele apita.

2.5

Testes data driven

O problema

Cinco cenários de login inválido.

Você copia e cola o teste cinco vezes, mudando o e-mail e a mensagem.

Agora, quando o seletor do botão mudar, **você abre cinco testes**.

Separe os dados

```
[
  {
    "cenario": "o e-mail não está cadastrado",
    "email": "naoexiste@qa.com",
    "senha": "treino123",
    "mensagemEsperada": "Email e/ou senha inválidos"
  },
  {
    "cenario": "a senha está incorreta",
    ...
  }
]
```

O teste, uma vez só

```
const cenarios = require('../../fixtures/login-invalido.json')

cenarios.forEach((caso) => {
  it(`deve exibir erro quando ${caso.cenario}`, () => {
    cy.visit('/login')
    if (caso.email) cy.get('[data-testid="email"]').type(caso.email)
    if (caso.senha) cy.get('[data-testid="senha"]').type(caso.senha)
    cy.get('[data-testid="entrar"]').click()

    cy.get('.alert').should('contain', caso.mensagemEsperada)
  })
})
```

A pegadinha

```
// ❌ Direto no describe
cy.fixture('login-invalido').then((casos) => {
  casos.forEach((caso) => {
    it(`...`, () => { })
  })
})
```

```
Cannot call `cy.fixture()` outside a running test.
```

`require` roda quando o arquivo é carregado.
`cy.fixture()` é comando — e comando só roda **dentro** de um teste.

Adicionar cenário virou adicionar linha no JSON

`require` para gerar testes

`cy.fixture()` para usar dados dentro de um teste

Um detalhe que rende discussão

A requisição manda	A API responde
Sem a chave `email`	`email é obrigatório`
`email: ""`	`email não pode ficar em branco`

Qual das duas o teste deve esperar?

A que o usuário vê

O teste valida comportamento observável,
não implementação

Segredos

```
cy.get('[data-testid="email"]').type(Cypress.env('usuarioAdmin'))
```

```
npx cypress run --env usuarioAdmin=outro@qa.com
```

Senha real nunca vai para o repositório.

Nem em fixture, nem em constante, nem "só por enquanto".

A do admin do ServeRest é exceção: aplicação de estudo.

E o aviso amarelo no terminal?

O Cypress 15 depreciou o `Cypress.env()`. Funciona, mas vai sumir.

Para	Use
Valor público (URL da API)	`Cypress.expose('apiUrl')`
Segredo (senha)	`cy.env(['senhaAdmin'])` — valor chega no `.then()`

O conceito não muda: configuração fora do código, segredo fora do repositório.

2.6

Comandos customizados

O problema

O mesmo bloco de seis linhas aparece em doze arquivos.

Quando ele precisa mudar, você abre doze arquivos.

Comando simples

```
// cypress/support/commands.js
Cypress.Commands.add('login', (email, senha) => {
  cy.visit('/login')
  cy.get('[data-testid="email"]').type(email)
  cy.get('[data-testid="senha"]').type(senha)
  cy.get('[data-testid="entrar"]').click()
})
```

```
cy.login('admin.treinamento@qa.com', 'treino123')
```

A porta dos fundos, para todos

A função `entrarViaApi` do bloco 2.4 vira o comando **`cy.loginViaApi()`**:

```
cy.loginViaApi()                          // sem argumento: o admin da configuração
cy.loginViaApi(usuario.email, usuario.password)
```

Agora qualquer teste, de qualquer arquivo, entra sem passar pela tela.

Comando filho

```
Cypress.Commands.add('validarTexto',
  { prevSubject: true },
  (elemento, texto) => {
    cy.wrap(elemento).should('contain', texto)
  }
)
```

```
cy.get('h1').validarTexto('Bem Vindo')
```

`prevSubject: true` = *"eu recebo o elemento anterior"*

O padrão dos valores padrão

```
Cypress.Commands.add('criarUsuarioViaApi', (dados = {}) => {
  const usuario = {
    nome: dados.nome || 'Usuário Treinamento',
    email: dados.email || `user_${Date.now()}@qa.com`,
    password: dados.password || 'teste123',
    administrador: dados.administrador || 'true'
  }

  return cy.request('POST', `${Cypress.env('apiUrl')}/usuarios`, usuario)
    .then((res) => ({ ...usuario, _id: res.body._id }))
})
```

Atende os dois extremos

```
// não me importa quem, só quero um usuário
cy.criarUsuarioViaApi()

// me importa o nome, o resto tanto faz
cy.criarUsuarioViaApi({ nome: 'Nome Específico' })
```

Se o comando exigir sete parâmetros

as pessoas vão escrever
o `cy.request` na mão

O que ele devolve

```
return cy.request(...).then((res) => ({ ...usuario, _id: res.body._id }))
```

O objeto **com o** `_id`. É isso que permite encadear:

```
cy.criarUsuarioViaApi().then((usuario) => {
  cy.loginViaApi(usuario.email, usuario.password)
})
```

Onde está o limite?

Quando vira demais

```
cy.preencherFormularioDeProduto(...)
cy.validarTabelaDeUsuarios(...)
cy.clicarNoBotaoDeCadastroDeUsuario(...)
```

Um comando por tela → `commands.js` vira um depósito de duzentas linhas e ninguém acha nada.

Comando é para ação transversal.

Vocês vão sentir esse incômodo hoje

Guardem

Amanhã a gente resolve

2.7

Evidências e plugins

Evidências

```
{
  video: true,
  videoCompression: 32,
  screenshotOnRunFailure: true,
  trashAssetsBeforeRuns: true
}
```

- Vídeo só é gerado em `cypress run` — e vem desligado por padrão desde o Cypress 13
- Vídeo pesa e desacelera — muita equipe guarda só o das falhas
- Screenshot de falha é automático

Por que plugins existem?

A ponte para fora

O teste roda **dentro** do navegador.

E o navegador, por segurança, não tem acesso a:

- sistema de arquivos
- banco de dados
- terminal

Plugin é código Node que roda fora.

Instalando um: faker

```
npm install --save-dev @faker-js/faker@10.6.0
```

```
import { faker } from '@faker-js/faker/locale/pt_BR'

const usuario = {
  nome: faker.person.fullName(),
  email: `${faker.internet.username()}_${Date.now()}@qa.com`
}
```

Repare no `Date.now()` mesmo usando faker.

Faker gera dados realistas

Não **únicos**

Realismo não substitui unicidade

Plugins que vale conhecer

Plugin	Para quê
`cypress-mochawesome-reporter`	Relatório HTML
`@faker-js/faker`	Massa realista
`cypress-real-events`	Hover e teclado reais
`cypress-plugin-api`	Visualização de requisições
`cypress-axe`	Acessibilidade

Upload de arquivo **não** precisa de plugin: `.selectFile()` é nativo desde o Cypress 9.3.

2.8

Lab

Lab do Dia 2 · 40 min

API

Criar usuário admin via comando · autenticar e cadastrar produto · validar na listagem

Interface

Entrar via API · validar o produto na tela · interceptar a listagem

Data driven

Fixture com ao menos três cenários · gerar os testes em loop

Requisitos técnicos

- Ao menos **dois comandos customizados**
- Ao menos **um alias de requisição**
- **Nenhum** `cy.wait` numérico
- Massa criada pelo próprio teste

Comandos disponíveis

```
cy.criarUsuarioViaApi({ nome, email, password, administrador })
cy.cadastrarProdutoViaApi({ nome, preco, descricao, quantidade })
cy.loginViaApi(email, senha)
cy.obterToken(email, senha)
```

O que você leva do Dia 2

1. **Alias de requisição** transforma espera em fato
2. **API é mais rápida e estável** — use para preparar estado
3. `failOnStatusCode: false` para o caminho negativo
4. `cy.intercept` **simula o impossível** — e acha bug de verdade
5. **Login via API** como setup padrão
6. **Fixture +** `forEach` em vez de copiar e colar
7. **Valores padrão** tornam o comando reutilizável

Uma pergunta para levar para casa

Abra os arquivos de ontem e de hoje

Procure por:

```
[data-testid="entrar"]
```

Quantas ocorrências?

Se renomearem esse atributo amanhã

quantos arquivos você vai precisar abrir?

Até amanhã.